



# 北京航空航天大学

BEIHANG UNIVERSITY

## OpenMP 编程实验报告

《并行程序设计 A》课程报告

学 院： 软件学院

班 级： ZY25211

姓 名： 刘益洲

学 号： ZY2521110

# 目录

|                          |   |
|--------------------------|---|
| 一、 实验环境 .....            | 1 |
| 二、 简介 .....              | 2 |
| 三、 实验一：并行矩阵乘法 .....      | 2 |
| (一) 实验内容与方法 .....        | 2 |
| (二) 实验过程.....            | 2 |
| (三) 实验结果与分析 .....        | 4 |
| 四、 实验二：正弦函数的泰勒级数计算 ..... | 5 |
| (一) 实验内容与方法 .....        | 5 |
| (二) 实验过程.....            | 5 |
| (三) 实验结果与分析 .....        | 8 |

## 一、实验环境

本实验在表 1 所列硬件与系统环境下完成，以下信息由本机在实验数据采集时通过 `system_profiler` 与 `sysctl` 读取。

表 1 实验用机硬件与操作系统环境

| 项目      | 规格或说明                                    |
|---------|------------------------------------------|
| 机型名称    | MacBook Pro                              |
| 机型标识    | Mac16,8                                  |
| 处理器     | Apple M4 Pro; 共 12 核 (8 个性能核心 + 4 个能效核心) |
| 逻辑处理器数量 | 12                                       |
| 内存容量    | 24 GB                                    |
| 图形处理器   | Apple M4 Pro 集成 GPU (16 核, 支持 Metal 3)   |
| 操作系统    | macOS 15.4.1 (构建号 24E263)                |

编译、链接与 OpenMP 运行时所依赖的主要软件版本见表 2 (由本机在实验环境整理阶段通过终端输出与预处理宏查询得到)。

表 2 编译器与 OpenMP 相关软件环境

| 项目              | 规格或说明                                                                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| C/C++ 编译器       | Apple clang 17.0.0 (clang-1700.0.13.5; Xcode 默认工具链)                                                                                                 |
| 编译目标三元组         | arm64-apple-darwin24.4.0                                                                                                                            |
| 构建工具            | GNU Make 3.81                                                                                                                                       |
| OpenMP 运行时      | Homebrew 公式 libomp, Cellar 版本 21.1.3 (LLVM OpenMP 运行时)                                                                                              |
| 编译期 OpenMP 接口级别 | 预处理宏 <code>_OPENMP</code> 为 202011 (对应 OpenMP 5.1)                                                                                                  |
| 编译与链接要点         | 使用 <code>-O3</code> 优化; OpenMP 源文件采用 <code>-xpreprocessor</code> <code>-fopenmp</code> ; 链接时使用 <code>-lomp</code> , 头文件与库由上述 Homebrew libomp 安装路径提供 |

## 二、简介

这里是简介。

## 三、实验一：并行矩阵乘法

### （一）实验内容与方法

实验内容为使用 OpenMP 编写并行程序，计算稠密方阵乘积  $C = AB$ 。矩阵为方阵，阶数  $N$  不小于 8000，元素类型为双精度浮点数；题目中关于规模较大时内存需求约在 480 MB 量级的说明，随实际参与存储的矩阵份数而变。按照课程要求，还需在多核环境下针对多种线程配置，与串行程序对比并给出加速比，以便分析并行化收益。

基本做法如下。在相同初始化数据下分别实现无并行指导的串行核函数与带 OpenMP 并行 for 的版本；乘法本身用三重循环完成，并行度加在最外层行循环上，矩阵数据在堆上动态分配以避免阶数较大时栈空间不足。在表 1、表 2 的环境中，对不同线程数分别运行程序，对矩阵乘法核心段计时（如使用 `omp_get_wtime`），以串行时间与各并行时间之比作为加速比，比较线程数变化对结果的影响。

### （二）实验过程

实验代码写在 `matmul_omp.c` 中，与同目录的 `Makefile` 配套使用。执行 `make` 生成可执行文件 `matmul_omp`；编译选项与链接方式即前文实验环境表中给出的 Apple clang 与 Homebrew libomp 组合，此处不重复抄录。若可执行文件已过期或更换了源码，应先重新编译再测时间。

运行时的两个命令行参数分别为阶数  $N$  与线程控制量。后者取 0 时只调用普通三重循环的串行核，进程中不出现并行区；取正整数时则先设定 OpenMP 线程数，再进入最外层带 `parallel for` 的同一算式。这样可以把「纯串行」与「含 OpenMP 运行时的并行」分开测量，其中并行分支在线程数为 1 时仍可用来观察单线程下的 OpenMP 开销。主程序在完成三块矩阵的堆分配与初始化后，用一对 `omp_get_wtime` 包住上述乘法调用，算得一次运行的 `wall time`，并以一行文本向标准输出报告  $N$ 、线程参数与秒数，供后续批处理汇总。

下面两段摘录取自当前源文件，对应串行核、并行核以及主程序中的计时段落。两套内核的算术相同，区别只在于最外层循环是否交给 OpenMP 切分；并行指导里选用 `schedule(static)`，并通过 `default(none)` 与 `shared` 列出对矩阵指针与阶数的共享约定。

```

1 static void matmul_serial(const double *A, const double *B, double *C, int n) {
2     for (int i = 0; i < n; i++) {
3         for (int j = 0; j < n; j++) {
4             double sum = 0.0;
5             for (int k = 0; k < n; k++) {
6                 sum += A[(size_t)i * (size_t)n + (size_t)k] *
7                     B[(size_t)k * (size_t)n + (size_t)j];
8             }
9             C[(size_t)i * (size_t)n + (size_t)j] = sum;
10        }
11    }
12 }
13
14 static void matmul_omp(const double *A, const double *B, double *C, int n,
15                        int num_threads) {
16     omp_set_num_threads(num_threads);
17 #pragma omp parallel for default(none) shared(A, B, C, n) schedule(static)
18     for (int i = 0; i < n; i++) {
19         for (int j = 0; j < n; j++) {
20             double sum = 0.0;
21             for (int k = 0; k < n; k++) {
22                 sum += A[(size_t)i * (size_t)n + (size_t)k] *
23                     B[(size_t)k * (size_t)n + (size_t)j];
24             }
25             C[(size_t)i * (size_t)n + (size_t)j] = sum;
26         }
27     }
28 }

```

代码 1: matmul\_omp.c 中串行核与并行核 (节选)

```

1     init_matrices(A, B, n);
2
3     double t0 = omp_get_wtime();
4     if (threads == 0) {
5         matmul_serial(A, B, C, n);
6     } else {
7         matmul_omp(A, B, C, n, threads);
8     }
9     double t1 = omp_get_wtime();
10    const double elapsed = t1 - t0;
11
12    printf("%d %d %.9f\n", n, threads, elapsed);

```

## 代码 2: matmul\_omp.c 中计时与按线程参数分支 (节选)

为在  $N = 8000$  下连续得到多组线程设置的时间, 另外写了 shell 脚本 `run_matmul_bench_8000`。它在脚本所在目录按固定顺序用参数 0、2、4、8 调用 `matmul_omp`, 并把每次的标准输出追加到同一文本里; 默认结果文件名为 `matmul_bench_8000.txt`, 脚本开头会清空该文件再写入本轮数据, 以免与旧记录混在一起。本机在实验环境表中所列处理器的逻辑处理器数量有限, 当 OpenMP 线程数超过能够同时占用不同逻辑核心、真正并行执行的最大规模后, 再增加线程主要带来调度与抢占, 并不能再增加「同时在做乘法」的硬件并行度, 因此未再尝试高于 8 的线程配置, 而是把 0、2、4、8 作为与可用并行度相匹配的一组对比点。

计时在条件尽量稳定的前提下独立重复了三遍: 每一遍均重新执行上述脚本, 分别得到四组线程各一次耗时, 将结果存成三份原始记录, 供之后核对。最终写入报告表格或绘制曲线的数值, 对三遍实验中同一设置下的耗时取算术平均, 以削弱单次测量中系统负载波动带来的偶然性。由于串行那一次往往占去单轮测量中的绝大部分时间, 完整三轮实验累计等待较长, 但操作仍属固定流程的重复执行, 无需在运行过程中人工干预。

### (三) 实验结果与分析

三轮测量的原始耗时及算术平均见表 3, 图 1 为基于均值的耗时与加速比示意。平均而言, 串行约 831.5 s, 2、4、8 线程分别约 432.2 s、229.9 s、132.0 s; 相对串行的加速比分别约 1.92、3.62 与 6.30, 明显低于线性理想值, 应与原存储访问方式、带宽及 OpenMP 开销等因素有关。

表 3 矩阵乘法: 各轮计时 (s) 与平均值 ( $N = 8000$ )

| 测量次序  | 线程 0   | 线程 2   | 线程 4   | 线程 8   |
|-------|--------|--------|--------|--------|
| 第 1 次 | 827.40 | 431.33 | 227.09 | 134.11 |
| 第 2 次 | 827.51 | 432.92 | 230.96 | 133.38 |
| 第 3 次 | 839.53 | 432.43 | 231.60 | 128.58 |
| 平均    | 831.48 | 432.22 | 229.89 | 132.02 |

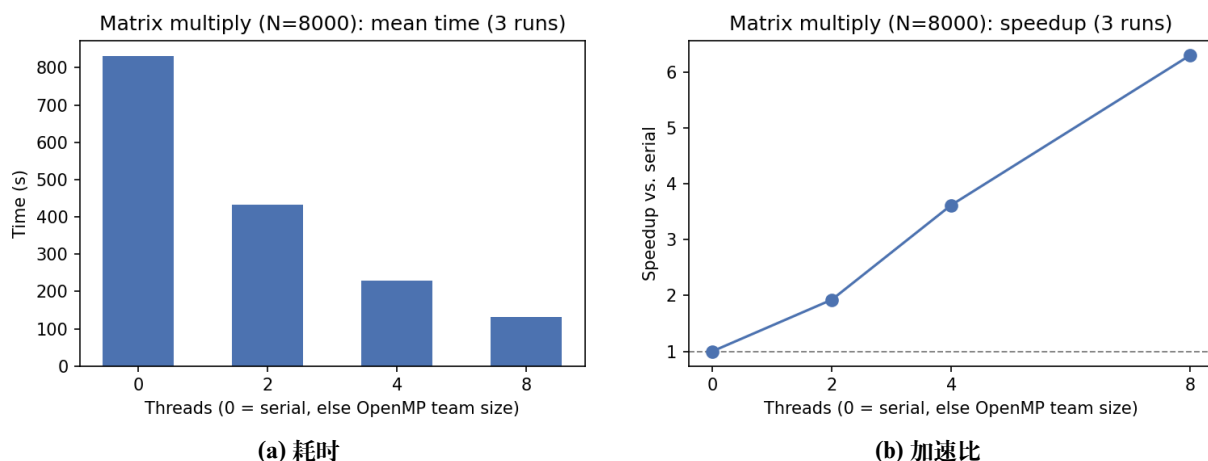


图1 实验一：矩阵乘法

整体仍表明在该规模与实现方式下 OpenMP 行并行能带来数倍量级的 wall time 缩短；继续增加线程若超出可用于真正并行的逻辑核数，收益会进一步受操作系统调度与资源争用制约，这与前文控制线程上界的选择相一致。

## 四、实验二：正弦函数的泰勒级数计算

### (一) 实验内容与方法

本实验要求编写程序，按泰勒级数计算正弦函数  $\sin x$ 。级数形式为

$$\sin x = \sum_{i=0}^{\infty} (-1)^i \frac{x^{2i+1}}{(2i+1)!}.$$

实现时只能在有限项上截断，故需选定截断项数或等价地设定精度准则，使部分和与真值之差在可接受范围内； $x$  在本实验中事先固定，全部工作量集中在对该  $x$  的级数求和上。

并行策略取为：在  $x$  与截断方式不变的前提下，将指标  $i$  所对应的各个级数项分摊到多个线程计算。每个线程各自累加所负责项的贡献，再在并行区末尾通过规约得到总部分和，从而得到  $\sin x$  的近似值。与实验一类似，课程要求在多核环境下以串行实现为参照，在不同 OpenMP 线程数下测量求和核心段的耗时并比较加速比。

### (二) 实验过程

程序写在 `sin_taylor_omp.c`，与同目录 `Makefile` 中的目标 `sin_taylor_omp` 对应；执行 `make sin_taylor_omp` 即可用与实验环境表一致的编译选项链接 OpenMP 与数学库。可执行文件接受三个参数，依次为自变量  $x$ 、截断项数以及线程控制量：取

0 时仅执行串行求和，取正整数时通过 `omp_set_num_threads` 进入并行分支，计时仍用 `omp_get_wtime` 包住核心段，并向标准输出打印一行「 $x$ 、项数、线程参数、耗时」，便于脚本收集。

实现上与正文所述策略对应的核心片段如下。第一项给出 `taylor_term_at`：用  $\log|x|$  与  $\lgamma((2i+2))$  写出第  $i$  项模长的对数，再 `exp` 回去并结合交替符号与 `copysign` 处理  $x$  的符号，使每个线程能在  $O(1)$  代价下得到其负责区间的首项，避免自 0 起逐项快进到 `istart` 所带来的总工作量随线程数病态放大。紧随其后的 `sin_taylor_serial` 为纯串行对照：同一递推系数在循环中更新 `term` 并累加 `sum`。

```

1  /* 第 i 项:  $(-1)^i * x^{(2i+1)} / (2i+1)!$ ，用于各线程区间起点，避免从  $j=0$  快进的  $O(n*$ 
   线程数) 冗余 */
2  static double taylor_term_at(double x, long i) {
3      const double sign_alt = (i & 1L) ? -1.0 : 1.0;
4      const double ax = fabs(x);
5      if (ax == 0.0) {
6          return (i == 0) ? x : 0.0;
7      }
8      const double di = (double)(2 * i + 1);
9      const double lfac = lgamma((double)(2 * i + 2));
10     double lm = di * log(ax) - lfac;
11     if (lm < -745.0) {
12         return 0.0;
13     }
14     const double mag = exp(lm);
15     return sign_alt * copysign(mag, x);
16 }
17
18 static double sin_taylor_serial(double x, long n) {
19     if (n <= 0) {
20         return 0.0;
21     }
22     double sum = 0.0;
23     double term = x;
24     for (long i = 0; i < n; i++) {
25         sum += term;
26         term *= -x * x / (double)((2 * i + 2) * (2 * i + 3));
27     }
28     return sum;
29 }

```

代码 3: `sin_taylor_omp.c`: 闭式单项与串行求和 (节选)



并行路径由独立并行区与对 `sum` 的 `reduction(+:...)` 完成合并：各线程按编号划分连续的项下标区间, 在区间内先用 `taylor_term_at(x, istart)` 定初值, 再用与串行相同的系数做递推累加到局部量 `partial`, 最后规约到共享意义上的总和。

```

1 static double sin_taylor_omp(double x, long n, int num_threads) {
2     if (n <= 0) {
3         return 0.0;
4     }
5     double sum = 0.0;
6     omp_set_num_threads(num_threads);
7 #pragma omp parallel default(none) shared(x, n) reduction(+ : sum)
8     {
9         const int tid = omp_get_thread_num();
10        const int nt = omp_get_num_threads();
11        const long chunk = (n + (long)nt - 1) / (long)nt;
12        const long istart = (long)tid * chunk;
13        const long iend = istart + chunk < n ? istart + chunk : n;
14
15        double partial = 0.0;
16        if (istart < iend) {
17            double term = taylor_term_at(x, istart);
18            for (long i = istart; i < iend; i++) {
19                partial += term;
20                term *= -x * x / (double)((2 * i + 2) * (2 * i + 3));
21            }
22        }
23        sum += partial;
24    }
25    return sum;
26 }

```

代码 4: `sin_taylor_omp.c`: OpenMP 并行求和 (节选)

主程序中对计时与分支的写法与实验一类似：一对 `omp_get_wtime` 包住串行或并行调用, 并将结果赋给 `volatile double` 以防被优化掉影响计时的语义。

```

1 double t0 = omp_get_wtime();
2 volatile double result;
3 if (threads == 0) {
4     result = sin_taylor_serial(x, n);
5 } else {
6     result = sin_taylor_omp(x, n, threads);
7 }
8 double t1 = omp_get_wtime();

```

```

9  (void)result;
10
11  printf("%.17g %ld %d %.9f\n", x, n, threads, t1 - t0);

```

代码 5: sin\_taylor\_omp.c: 计时与串行/并行分支 (节选)

批处理脚本为 `run_sin_taylor_bench.sh`。其中  $x$  与级数项数在脚本里写成固定常量，一轮测量中依次以线程数 0、2、4、8 调用 `sin_taylor_omp`，将四行输出写入 `results` 目录下的文本文件（默认文件名为 `sin_taylor_bench.txt`，每轮开始前清空该文件再写入本轮结果）。与实验一相同，本机逻辑处理器数目决定了真正能同时跑在不同核心上的线程上限，超过该上限后再加 **OpenMP** 线程并不能增加实质上的并行工作量，主要体现为时间片轮转，因此未再测更高的线程数。

计时同样独立重复了三遍：每一遍在相近的系统状态下完整执行上述脚本一次，保留三份原始输出，供事后核对。报告后续表格或曲线所采用的各配置耗时，对三遍结果取算术平均，以减弱偶发的后台负载波动。若某一轮中串行配置仍明显慢于并行配置，三轮合计等待时间仍会较长，但操作步骤仍是重复同一脚本，无需中途人工参与。

### （三）实验结果与分析

三轮原始数据及均值见表 4，图 2 基于均值绘制。平均意义下，串行约 226.9 s，2、4、8 线程分别约 116.3 s、61.3 s、32.4 s，加速比约 1.95、3.70、7.01；相对实验一，8 线程时略更接近按比例扩展的趋势，与求和算法形态及规约开销占比不同有关，但仍非理想线性。

表 4 Sin Taylor: 各轮计时 (s) 与平均值 (脚本内相同  $x$ 、项数)

| 测量次序  | 线程 0   | 线程 2   | 线程 4  | 线程 8  |
|-------|--------|--------|-------|-------|
| 第 1 次 | 230.10 | 118.66 | 62.73 | 31.83 |
| 第 2 次 | 223.31 | 114.55 | 60.44 | 32.19 |
| 第 3 次 | 227.28 | 115.63 | 60.74 | 33.05 |
| 平均    | 226.90 | 116.28 | 61.30 | 32.36 |

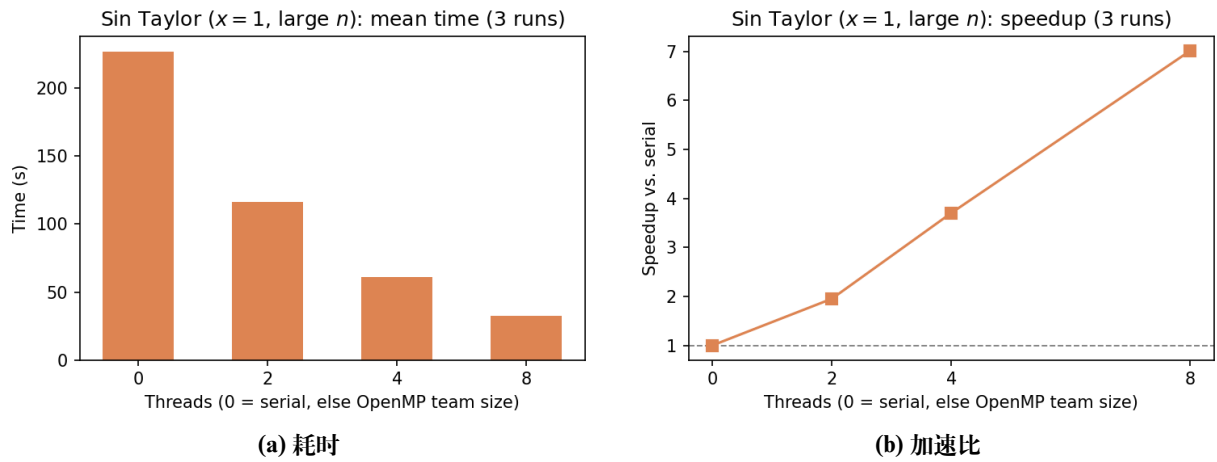


图 2 实验二：Sin Taylor

综合来看，两类算子在多核上均获得了可观的加速，但受算法与硬件共同约束，加速比曲线均呈明显「边际递减」；要在更高端数或更大线程规模下进一步挖潜，需要结合数值稳定性、负载均衡策略以及是否引入更缓存友好的计算排列另行设计，这已经超出本实验所用的最简 OpenMP 行并行与项划分方案。